

HelixMemory

HelixMemory

Адзін мазгавы блок памяці для агентаў AI — чатыры найлепшыя рухавікі, злітыя ў адно.

Зводка

HelixMemory — гэта Go SDK, які аб'ядноўвае чатыры вядучыя сістэмы памяці (Mem0, Cognee, Letta, Graphiti) у адзіны кагнітыўны рухавік памяці, што ажыццяўляе іх паралельны пошук і злівае вынікі. Ён забяспечвае AI-дадаткам адзін трывалы, дэдублікаваны і пераранжыраваны слой памяці замест чатырох разрозненых.

Кароткае апісанне

HelixMemory — гэта Go SDK, які злучае Mem0, Cognee, Letta і Graphiti у адзіны кагнітыўны рухавік памяці для AI-дадаткаў. Ён разумна размеркаваўвае запісы, паралельна шукае ў кожным з рухавікоў і злівае вынікі праз трохэтапны канвеер збору-дэдублікацыі-пераранжыроўкі.

Падрабязнае апісанне

HelixMemory — гэта адзіны кагнітыўны рухавік памяці для AI-дадаткаў, які пастаўляецца як Go SDK (модуль `digital.vasic.helixmemory`, Go версіі 1.25+). Яго асноўная здагадка заключаецца ў тым, што ніводзін праект памяці ніколі не будзе лепшым ва ўсім — таму замест таго, каб перапісваць памяць з нуля і атрымліваць у спадчыну абмежаванні аднаго праекта, ён аркеструе чатыры найлепшыя сістэмы, дазваляючы кожнай праявіць свае моцныя бакі: Mem0 для дынамічнага вымання фактаў і кіравання перавагамі, Cognee для семантычных ведаў на аснове канвеераў ECL, Letta

для агенцкага рантайму з рэдагавальнай памяццю і вылічэннямі падчас сну, а таксама Graphiti для бітэмпарнага графа ведаў, які аналізуе змены фактаў з цягам часу.

Рухавік зліцця — гэта тое, што ператварае чатыры незалежныя сховішчы ў адзін мазгавы блок. Падчас запісу кожная ўваходная інфармацыя класіфікуецца па зместу і накіроўваецца ў той рухавік, які найлепш падыходзіць для яе захоўвання. Падчас чытання запыт паралельна распаўсюджваецца па ўсіх рухавіках, а сырыя вынікі паступаюць у трохэтапны канвеер зліцця — збор, дэдублікацыя, пераранжыроўка па крыніцах — каб карыстальнік атрымліваў не чатыры шумныя і перакрываючыся наборы вынікаў, а адзін чысты і ранжыраваны адказ. Колавая выключальнікі ахопліваюць кожны рухавік для эlegantнага зніжэння прадукцыйнасці: калі адзін з іх адключаецца, яго выключальнік спрацоўвае, а астатнія працягваюць працаваць, не цягнуць за сабой увесь слой памяці. Паколькі рухавік рэалізуе інтэрфейс MemoryStore, ён лёгка ўстаўляецца замест звычайнага пастаўшчыка памяці — без перабудовы кліенцкага кода — а метрыкі Prometheus адлюстроўваюць унутраныя працэсы маршрутызацыі і зліцця для поўнай назіральнасці.

HelixMemory быў створаны як слой памяці для HelixAgent — шырэйшага ансамбля Helix AI — і пераносіць у вобласць памяці дысцыпліну антыблефавай праверкі, уласцівую гэтаму сямейству: убудаваны тэсціроўшчык выпрабоўвае рэальныя вытворчыя шляхі — маршрутызацыю, зліццё, трансляцыю, каловы выключальнік — а паралельны мутатар наўмысна парушае інварыянты, каб пацвердзіць, што тэсты сапраўды правальваюцца, калі лагіка парушана, і таму зялёны набор тэстаў мае рэальны сэнс.

Чаму мы яго стварылі

Змест

Агенты AI патрабуюць даўгавечнай і якаснай памяці, але экасістэма фрагментаваная — кожны праект памяці (Mem0, Cognee, Letta, Graphiti) моцны ў адным і слабы ў іншым. HelixMemory быў створаны, каб даць HelixAgent адзіную паверхню памяці, якая аб'ядноўвае іх перавагі, не прымушаючы пры гэтым прывязвацца да аднаго з іх.

Чаму гэта змяняе правілы гульні

Гэта ліквідуе прымусовы выбар. Чатыры сістэмы памяці, якія звычайна канкуруюць за адзін і той жа слот, становяцца дадатковымі бэкэндамі за адзіным інтэрфейсам — так дадатак атрымлівае дынамічнае выманне фактаў, семантычныя графі ведаў, памяць агента з станам і бі-часавы лагічны аналіз адначасова, з аўтаматычнай дэдублікацыяй і пераранжыроўкай крыніц. Тое, што раней было непрактычна, — разглядаць пытанне «які рухавічок памяці нам абраць?» як ілжывую дылему: HelixMemory дазваляе атрымаць усе іх перавагі адразу, праз адзін убудаваны MemoryStore, не атрымліваючы пры гэтым слабых месцаў якога-небудзь рухавіка і не прывязваючыся да яго.

Што новага

- **Мультибэкэндавае зліццё** (збор → дэдублікацыя → пераранжыроўка крыніц), якое вяртае адзін ранжыраваны набор вынікаў замест таго, каб прымацоўваць кліента да аднаго сховішча.
- **Разумнае маршрутызаванне запісаў**, якое класіфікуе кожную памяць па зместу і адпраўляе яе ў той рухавік, які найлепш падыходзіць для яе захоўвання, каб правільныя дадзеныя апынуліся ў правільным сховішчы.
- **Плыўная дэградацыя** праз схемы абароны асобных бэкэндаў — няспраўны рухавік ізалюецца, а не прыводзіць да краху, і астатнія працягваюць працаваць.
- **Кансалідацыя падчас сну** (праз Letta), якая перапрацоўвае памяць у перыяды бяздзейнасці замест таго, каб рабіць гэта толькі падчас запытаў.
- **Антыблефная праверка**: тэставы запуск на рэальным прадукцыйным кодзе ў спалучэнні з абгорткай мутацый, якая павінна правальвацца, калі парушаецца інварыянт, — гэта даказвае, што тэставы бар'ер з'яўляецца сапраўднай праверкай, а не таўталогіяй.

Найбольшыя тэхнічныя выклікі і як мы іх пераадолені

- **Аб'яднанне чатырох разнастайных бэкэндаў у адзін узгоднены набор вынікаў** — кожны рухавік вяртае памяць у сваім фармаце, і наіўнае аб'яднанне прыводзіць да дублікатаў і несумяшчальных ранжыроўкаў. Вырашана з дапамогай тыпізаванага рухавіка зліцця, які збірае дадзеныя з крыніц, выдаляе дублікаты і пераранжыруе ўсё на агульнай аснове, прычым інварыянт злітых падлікаў

правяраецца ў тэстах, каб аб'яднанне не магло ціха губляць ці падвойваць вынікі.

- **Захаванне працаздольнасці, калі адзін з бэкэндаў падае** — недаступны рухавік памяці не павінен спыняць усю сістэму. Вырашана з дапамогай схем абароны асобных бэкэндаў, якія працуюць паводле машыны станаў закрыты → адкрыты (пасля дасягнення парога адмоваў) → паўадкрыты (пасля тайм-аўту), ізалюючы няспраўны бэкэнд і працягваючы абслугоўванне праз астатнія, пакуль ён не адновіцца.
- **Доказ таго, што логіка памяці сапраўды працуе, а не проста кампілюецца** — з'ялены набор тэстаў бескарысны, калі тэсты не могуць праваліцца. Вырашана з дапамогай убудаванага тэставага запуску, які кіруе рэальным прадукцыйным кодам (маршрутызацыя, зліццё, перакладчык, схема абароны), і спаранай абгорткі мутацый, якая змяняе інварыянты і патрабуе, каб тэсты правальваліся, — такім чынам бар'ер правяраецца на сапраўднасць і не з'яўляецца таўталогіяй.

Змест

Тэхналагічны стэк

- **Go (1.25+)** — адзіны SDK і рантайм; абраны таму, што паралельнае чытанне з чатырох бэкэндаў з'яўляецца праблемай канкурэнтнасці, а гаруціны Go робяць яго танным, у той час як інтэрфейсныя тыпы даюць усёй сістэме адзіны чысты шво (MemoryStore), на якое могуць разлічваць карыстальнікі.
- **Mem0** — дынамічны бэкэнд для здабывання фактаў і кіравання перавагамі; выкарыстоўваецца для фрагмента памяці «што на самой справе аддае перавагу гэты карыстальнік / якія факты выплылі на паверхню».
- **Cognee** — семантычны бэкэнд на аснове графа ведаў, пабудаваны на канвеерах ECL; выкарыстоўваецца для захоўвання структураванага, звязанага ведаў замест плоскіх фактаў.
- **Letta** — бэкэнд з агенцкім рантаймам і рэдагаванымі блокамі памяці, а таксама вылічэннямі ў час сну; выкарыстоўваецца там, дзе памяць павінна захоўвацца як жывы стан агента і кансалідавацца ў перыяды бяздзейнасці.
- **Graphiti** — бітэмпарны бэкэнд на аснове графа ведаў; выкарыстоўваецца для разважанняў пра тое, як факты і сувязі змяняюцца з цягам часу, а не толькі пра іх бягучую каштоўнасць.
- **PostgreSQL + Neo4j + Redis** — рэальныя сховішчы даных, на якіх працуюць бэкэнды, разгорнутыя для сапраўднага інтэграцыйнага тэсціравання праз make infra-start, каб

тэставы набор правяраў жывую інфраструктуру, а не макеты.

- **Prometheus** — метрыкі і назіральнасць, падлучаныя праз канвеер зліцця, каб маршрутызацыя і паводзіны зліцця маглі вымярацца ў прадукцыйным асяроддзі, а не заставаліся чорнай скрыняй.
- **Шво перакладчыка інтэрнацыяналізацыі** — прастора імёнаў (helixmemory_) для паверхні радкоў, якая захоўваецца, каб любы будучы карыстальніцкі слой мог быць лакалізаваны без пераробкі ядра.

Стан і заўвагі аб шчырасці

- **Стан: бэта.** Працуючы SDK; створаны як слой памяці для HelixAgent.
- **Ліцэнзія: не вызначана.** Ліцэнзія не была выяўлена праз GitHub API — НЕПРАВЕРАНА / не абвешчана.
- Адлюстраваная назва «HelixMemory» адпавядае рэпазіторыю memory. Паказчыкі дакладнасці, прыведзеныя ў README, з'яўляюцца заявамі пастаўшчыкоў, а не вымярэннямі HelixMemory, і тут не падаюцца.

Прыярытэтны ўзровень: Helix-прымарны.